

1. Write a pytest test function that checks whether the sum of two numbers (3 and 5) equals 8.

```
def test_sum():  
    assert 3 + 5 == 8
```

2. Create a pytest test that intentionally fails by asserting that "hello".upper() equals "hello".

```
def test_fail():  
    assert "hello".upper() == "hello"
```

3. Define a pytest fixture that returns a list of numbers [1, 2, 3]. Write a test that uses this fixture to verify the list length is 3.

```
import pytest  
  
@pytest.fixture  
def sample_list():  
    return [1, 2, 3]  
  
def test_list_length(sample_list):  
    assert len(sample_list) == 3
```

4. Use @pytest.mark.parametrize to test a function square(x) for inputs 2, 3, 4 and expected outputs 4, 9, 16.

```
import pytest  
  
def square(x):  
    return x * x  
  
@pytest.mark.parametrize("input, expected", [  
    (2, 4),  
    (3, 9),
```

```
    (4, 16)
])
def test_square(input, expected):
    assert square(input) == expected
```

5. Write a pytest test that verifies a ZeroDivisionError is raised when dividing by zero using pytest.raises.

```
import pytest

def test_zero_division():
    with pytest.raises(ZeroDivisionError):
        x = 10 / 0
```